

Logique métier, page par page

Pour chaque page du cockpit : l'endpoint appelé, ses paramètres, la chaîne de fonctions traversée, et les maths derrière le calcul. Chaque section finit par la réponse à donner à l'oral.

Chiffres du moteur, seed 42, 100 000 années simulées. Toute valeur citée ici sort de deck/figures.json, écrit par le pipeline.

Vue d'ensemble

Page	Endpoint	Paramètres	Coût
Overview	les 4 ci-dessous	—	tout en cache
Telemetry	GET /api/telemetry/summary/ GET /api/frequency/	aucun	cache
Frequency	GET /api/frequency/ GET /api/assets/	seuil, fenêtre	~1 s
Severity	GET /api/severity/	aucun	cache
Simulation	POST /api/simulate/	7 (voir §4)	quelques s

- **Une seule règle d'architecture** : la vue lit la requête, appelle le moteur, sérialise. Aucun calcul de risque dans Django, aucun dans le front.
- **Deux niveaux de cache** : `get_dataset()` charge et ajuste une fois pour toutes (les CSV ne changent pas) ; `get_frequency()` et `get_simulation()` sont mémoïsés par jeu de paramètres.
- **Tout objet de résultat expose** `to_explanation()` — la trace numérotée que la CLI imprime, que l'API sert et que l'interface affiche. Une seule version de chaque chiffre.

1. Telemetry — ce que les sondes ont vu

Endpoint et paramètres

GET /api/telemetry/summary/ (aucun paramètre)
GET /api/frequency/

- Le second appel ne sert qu'au **dernier étage de l'entonnoir** : le taux d'incidents que la simulation consomme.
- Aucun paramètre : la normalisation ne dépend d'aucune convention.

Chaîne de fonctions

1. `load_assets()` — lit le référentiel des 20 actifs.
2. `load_siem()` / `load_edr()` — lisent chaque export et le traduisent en `SecurityEvent` canoniques. C'est là que les deux échelles de sévérité sont ramenées à un vocabulaire commun.
3. `severity_from_siem_label()` — Low/Medium/High/Critical → classe.
4. `severity_from_edr_risk()` — score 0–999 → classe, par points de coupure.
5. `merge_feeds()` — **dédoublonne** sur l'union des deux flux et produit le `NormalizationReport`.
6. `observed_window()` — déduit la fenêtre d'observation des événements eux-mêmes.
7. `summarize_telemetry()` — agrège en buckets hebdomadaires, mix de sévérité, top techniques.

Les maths

- **Normalisation des sévérités.** Le SIEM donne un libellé, l'EDR un score. Points de coupure EDR : < 50 low, 50–69 medium, 70–93 high, ≥ 94 critical. Score numérique associé à chaque classe : 0,25 / 0,5 / 0,75 / 1,0.
- **Clé de déduplication** : le triplet (`asset_id`, `technique`, `observed_at`). Deux rapports portant le même triplet décrivent le même événement réel.
- **Dédoublonnage sur l'union, pas par jointure.** Chaque flux répète déjà des clés en interne (761 fois côté SIEM, 543 côté EDR) ; une jointure interne multiplierait ces doublons et renverrait 13 055 lignes au lieu des 12 343 vrais recoupements.
- **Règle de fusion : la pire sévérité gagne.** Si l'EDR voit critical et le SIEM medium, on garde critical — l'outil le plus proche du poste a vu quelque chose. Prendre la moyenne diluerait un signal fort.
- **Facteur d'annualisation** = 365 / jours observés = 365 / 212 = **1,7217** (1^{er} nov. 2025 → 31 mai 2026). Recalculé depuis les données, jamais écrit en dur.

Chiffres produits

Grandeur	Valeur	D'où elle vient
Lignes brutes	45 840	somme des deux exports
Événements distincts	32 193	après déduplication
Doublons absorbés	13 647	dont 12 343 inter-flux
Inflation évitée	42,4 %	ce qu'une concaténation naïve aurait ajouté

À DIRE Les deux outils ne sont pas complémentaires, ils sont partiellement redondants : 12 343 événements portent le même actif, la même technique et le même horodatage des deux côtés. Je dédoublonne sur ce triplet en gardant la pire sévérité, ce qui évite 42,4 % d'inflation sur toutes les fréquences en aval.

2. Frequency — à quelle fréquence on est attaqué

Endpoint et paramètres

GET /api/frequency/?severity_threshold=high&session_window_hours=24

GET /api/assets/?severity_threshold=high&session_window_hours=24

Paramètre	Défaut	Effet
severity_threshold	high	sévérité minimale pour compter comme attaque
session_window_hours	24	silence qui ouvre un nouvel épisode

Ces deux paramètres sont des conventions, pas des mesures. Ils sont exposés comme curseurs dans l'interface plutôt que cachés dans le code.

Chaîne de fonctions

1. `get_frequency(seuil, fenêtre)` — mémorisé par couple de paramètres.
2. `estimate_frequency()` — orchestre les quatre étapes ci-dessous.
3. `is_attack_grade(event, seuil)` — filtre : ne garde que high et au-delà.
4. `sessionize(events, params)` — regroupe les événements en **épisodes**.
5. `attack_type_for(technique)` — classe chaque technique MITRE en type d'attaque via un dictionnaire explicite ; l'épisode hérite du type de son événement le plus grave.
6. `peer_weighted_base_rate(incidents, params)` — taux d'incidents par organisation-année chez les pairs.
7. `calibrate(lambda_detected, base_rate)` — ajuste `p_materialize` pour raccorder les deux unités.

Les maths

- **Règle d'épisode.** Groupement par **actif seul** — un intrus déclenche les détections qu'il rencontre, et compter chaque type séparément recompterait la même intrusion sous plusieurs noms. On coupe dès qu'un silence dépasse la fenêtre, mesuré par rapport à l'événement **précédent** et non au premier, sinon une intrusion de trois jours serait découpée artificiellement.
- **λ détecté** = épisodes $\times$ 365 / jours observés = $911 \times 1,7217 = 1\,568$ / an.
- **Taux de base des pairs** = incidents pondérés $\div$ organisations-années pondérées. Numérateur et dénominateur portent le même noyau de similarité, un enregistrement par `company_id` — soit **1 310 organisations**.
- **Le changement d'unité, le point clé.** La télémétrie compte des attaques *détectées*, la base externe des incidents *à perte*. Ce sont deux unités différentes.
- **Calibration** : $\lambda_{\text{incident}} = \lambda_{\text{détecté}} \times p$, avec p fixé pour que $\lambda_{\text{incident}}$ égale le taux des pairs. Ici $p \approx 1$ détection sur 5 138, soit $\lambda_{\text{incident}} = 0,31$ / an.
- **Le mix vient de la télémétrie, le niveau de la base.** Chaque type reçoit $\lambda_{\text{incident}} \times (\text{ses épisodes} / \text{épisodes totaux})$.

Chiffres produits

Grandeur	Valeur	Calcul
Événements attack-grade	10 126	sévérité $\geq$ high
Épisodes	911	compression 11,1×
λ détecté	1 568 / an	épisodes $\times$ 365/212
λ incident	0,31 / an	λ détecté $\div$ 5 138

À DIRE Une alerte n'est pas une attaque : une intrusion produit une rafale de détections. Je regroupe par actif avec une fenêtre de silence de 24 h, ce qui ramène 10 126 événements attack-grade à 911 épisodes, soit 1 568 attaques détectées par an. Mais la télémétrie compte des détections et la base externe des incidents à perte — deux unités différentes. Je les raccorde par une calibration : une détection sur 5 138 finit en perte, soit 0,31 incident par an ancré sur 1 310 organisations comparables.

3. Severity — ce que coûte un incident

Endpoint et paramètres

GET /api/severity/ (aucun paramètre)

Aucun paramètre : le modèle de sévérité ne dépend que du profil cible, qui est fixe pour ce cas (Retail, ETI, maturité 55).

Chaîne de fonctions

1. `load_incidents()` — lit et **nettoie** la base externe ; renvoie aussi le `CleaningReport`.
2. `repair_mojibake()` — répare le double encodage UTF-8 des libellés de secteur, qui sépare « Énergie » en deux secteurs distincts.
3. `fit_severity_model()` — orchestre les étapes ci-dessous, une fois pour l'ensemble puis une fois par type d'attaque.
4. `peer_weights(incidents, params)` — un poids par incident, via `sector_weight()`, `size_weight()` et `maturity_weight()`.
5. `effective_sample_size(weights)` — n_{eff} de Kish, ce que la pondération coûte.
6. `fit_lognormal(losses, weights)` — maximum de vraisemblance pondéré sur les logs.
7. `diagnose()` — assemble les preuves *contre* l'ajustement : `weighted_ks()`, `qq_points()`, `fit_pareto_tail()`, `distribution_plot()`.

Les maths

- **Nettoyage.** Les pertes à -1 sont des valeurs **manquantes**, jamais des 0 €. Deux lignes concernées, exclues de l'ajustement : il reste **1 598 incidents exploitables sur 1 600**.
- **Pondération douce, jamais de filtre dur.** Un poids continu par incident, produit de trois facteurs de ressemblance : secteur (1,0 si Retail, sinon 0,4), taille (1,0 si ETI, sinon 0,6) et un noyau gaussien sur l'écart de maturité, de largeur $h = 15$.
- **Pourquoi pondérer.** Un filtre exact sur le profil ne laisse que **112 incidents sur 1 598**, et plus aucun type d'attaque n'a d'échantillon crédible. Mathématiquement propre, pratiquement inutilisable.
- **Ajustement lognormal pondéré**, sur l'échelle des logs — c'est simplement une moyenne et une variance pondérées :

$$\begin{aligned}\mu &= \sum w_i \ln x_i / \sum w_i \\ \sigma^2 &= \sum w_i (\ln x_i - \mu)^2 / \sum w_i\end{aligned}$$

- **Retour en euros** : médiane = e^{μ} et moyenne = $e^{(\mu + \sigma^2/2)}$. Attention : μ et σ **ne sont pas** la moyenne et l'écart-type de la perte, mais ceux de son logarithme.
- **Exemple, supply chain** : $\mu = 12,119$, $\sigma = 2,582 \rightarrow$ médiane **183 k€**, moyenne **5,1 M€**. La queue multiplie la moyenne par **28** : $e^{(\sigma^2/2)}$.
- **C'est la moyenne qui alimente la perte annuelle**, pas la médiane. Un décideur qui raisonne sur « l'incident typique » se trompe d'un facteur 28.
- **n_{eff} de Kish** = $(\sum w)^2 / \sum w^2$. C'est le nombre d'observations de poids égal qui porteraient autant d'information. Il mesure la **régularité** des poids, pas leur taille. En dessous de **30**, le type bascule sur la distribution poolée, et la substitution est enregistrée.
- **Preuves publiées avec chaque ajustement** : KS pondéré (ici **0,118** contre un seuil indicatif $\approx 0,19$), QQ-plot des log-pertes, et une **queue de Pareto ajustée en rivale** — elle gagne sur 5 ajustements sur 9, donc VaR 99 et TVaR 99 se lisent comme des bornes basses.

À DIRE Une perte est bornée à zéro, sans plafond, et s'étale sur quatre ordres de grandeur : c'est la signature d'une variable dont le logarithme est à peu près normal, donc lognormale. J'ajuste par maximum de vraisemblance pondéré sur les logs — concrètement une moyenne et une variance pondérées — un ajustement par type d'attaque, parce qu'une lognormale unique se fait rejeter par Kolmogorov-Smirnov. Les poids viennent d'un noyau de similarité sur secteur, taille et maturité : je pondère plutôt que je filtre, parce qu'un filtre exact ne laisserait que 112 incidents. Le prix de cette souplesse est un échantillon effectif plus petit, que je publie type par type, avec repli sur la distribution poolée en dessous de 30.

4. Simulation — ce que coûte une année

Endpoint et paramètres

```
POST /api/simulate/
{
  "n_years": 5000,          "seed": 42,
  "severity_threshold": "high", "session_window_hours": 24,
  "loss_cap_eur": null,      "curve_points": 160,
  "histogram_bins": 40,      "include_sensitivity": true
}
```

Paramètre	Défaut	Effet
n_years	5 000 (API) / 100 000 (CLI)	finesse de la queue ; borné à 200 000
seed	42	reproductibilité bit à bit
severity_threshold	high	convention de fréquence, cf. §2
session_window_hours	24	convention de fréquence, cf. §2
loss_cap_eur	99,9 ^e centile ≈ 23,5 M€	plafond de plausibilité par incident ; inf pour désactiver
curve_points	160	résolution des courbes OEP/AEP
histogram_bins	40	bins de l'histogramme (log)
include_sensitivity	true	grille 3×3 — neuf runs de plus

Chaîne de fonctions

1. `get_simulation(n_years, seed, seuil, fenêtre, cap)` — mémoisé ; **le cap fait partie de la clé de cache**, deux plafonds sont deux réponses.
2. `simulate()` — la boucle Monte Carlo, vectorisée par blocs d'années.
3. `rng.poisson(lam= $\lambda$ _type, size=bloc)` — le nombre d'incidents de chaque année.
4. `rng.lognormal(mean= $\mu$ , sigma=sigma, size=total)` — le prix de chaque incident.
5. `np.minimum(losses, cap)` — le plafond de plausibilité.
6. `np.bincount(years, weights=losses)` — replie les incidents dans leur année.
7. `summarize(annual_losses)` — AAL, médiane, VaR, TVaR, P(aucune perte), maximum.
8. `exceedance_curve(series, kind)` — courbes AEP (total) et OEP (pire perte unitaire).
9. `SimulationResult.histogram(scale='log')` — bins log, années à zéro comptées à part.
10. `sensitivity_grid()` — rejoue seuil × fenêtre et rapporte l'AAL de chaque case.

Les maths

- **Loi de Poisson composée.** La perte annuelle est une somme d'un nombre aléatoire de termes aléatoires :
$$S = \sum_{i=1}^N X_i \quad \text{avec } N \sim \text{Poisson}(\lambda) \text{ et } X_i \sim \text{Lognormale}(\mu, \sigma)$$
- **Composer, pas multiplier.** Une année n'est pas « fréquence × coût moyen » : c'est zéro incident **73,7 % des années**, un parfois, deux très rarement — et la somme de ces cas n'est pas le produit des moyennes.
- **Pourquoi simuler et pas une formule.** L'espérance est analytique (formule de Wald : $E[S] = \lambda \times E[X]$), mais la **fonction de répartition de S n'a pas de forme fermée**. Or VaR 99 et TVaR 99 sont des quantiles de S. Il faut la distribution complète.
- **Ce qui fixe le nombre d'années** : la VaR 99 ne s'appuie que sur 1 % des années. Sur 100 000 années, cela fait 1 000 observations ; sur 1 000 années, dix — inutilisable. C'est la queue qui décide, pas la moyenne.
- **Métriques** : VaR_α = quantile d'ordre α ; $\text{TVaR}_\alpha = E[S \mid S \geq \text{VaR}_\alpha]$. $\text{TVaR} \geq \text{VaR}$ par construction — c'est une invariante testée.
- **AEP vs OEP.** AEP = total de l'année ; OEP = plus grosse perte unitaire de l'année. L'OEP ne peut jamais dépasser l'AEP à probabilité égale, et leur écart mesure la part des années à plusieurs incidents.

- **Plafond de plausibilité.** Une lognormale n'a pas de borne : à sigma 2,582 et sur 100 000 années, elle finit par tirer un incident coûtant plus que l'entreprise ne vaut (3,8 Md€ sans plafond). Chaque perte est donc écrêtée à **23,5 M€**, le 99,9^e centile des pertes réellement observées chez les pairs — 0,9 % des tirages sont concernés.
- **L'effet du plafond est rapporté, pas absorbé** : le résultat porte `draws_capped`, `draws_total` et `aal_uncapped`.

Chiffres produits

Mesure	Valeur	Lecture métier
AAL	274 k€	ligne budgétaire
Année médiane	0 €	73,7 % des années sans incident
VaR 95	816 k€	appétit au risque — 1 année sur 20
TVaR 95	4,8 M€	moyenne des 5 % pires années
VaR 99	6,7 M€	1 année sur 100
TVaR 99	15,1 M€	question de survie

Le rapport TVaR 95 / VaR 95 vaut 5,9 : une fois passé le seuil de la mauvaise année, la perte typique est presque six fois ce seuil. C'est la définition d'une queue lourde.

À DIRE Pour chaque année simulée je tire un nombre d'incidents par type dans une loi de Poisson, je prix chaque incident dans la lognormale de son type, et je somme. Cent mille années, graine 42, reproductible au centime. Je simule plutôt que d'appliquer une formule parce que l'espérance est analytique mais pas les quantiles : VaR 99 et TVaR 99 demandent la distribution complète. Et je plafonne chaque perte unitaire au 99,9^e centile des pertes observées chez les pairs, parce qu'une lognormale non bornée finit par tirer une année à 3,8 milliards pour une ETI de 1 200 personnes — le plafond retire 37,5 % de l'AAL, et je l'affiche plutôt que de l'absorber.

Aide-mémoire : cinq questions, cinq réponses de trente secondes

Question	Réponse
Pourquoi dédoublonner ?	12 343 événements sont vus par les deux flux. Concaténer gonflerait toutes les fréquences de 42,4 %.
Pourquoi des épisodes ?	Une intrusion produit une rafale d'alertes. Compter les alertes mesurerait le bavardage des sondes, pas la fréquence des attaques.
Pourquoi une calibration ?	La télémétrie compte des attaques détectées, la base externe des incidents à perte. Multiplier les deux directement est une erreur de catégorie — c'est ce qui donnait 12,5 Md€.
Pourquoi pondérer plutôt que filtrer ?	Le filtre exact ne laisse que 112 incidents et aucun type modélisable. La pondération garde tout le monde, les plus proches dominant.
Pourquoi Monte Carlo ?	L'espérance a une formule, les quantiles non. VaR 99 et TVaR 99 exigent la distribution complète de la loi composée.

Détail des concepts mathématiques : *CONCEPTS.md*. Décisions de modélisation et justifications : *METHODOLOGY.md*. Ce qui n'a pas été fait et pourquoi : *next_steps.md*.